**Software Analysis and Design | SWE5006 Team 1 — Lumen**

---

## UC1 Identity — Module Design and API

Sign up, sign in and manage the account

| Item | Detail |
|---|---|
| Assessment | Software Analysis and Design; Software Design Patterns (`SWE5006 Project Proposal - Lumen.docx` §6.3) |
| Use case realised | UC1 — sign up, sign in and manage the account (proposal §6.1) |
| Slice owner | LU YI. Pages 02 Sign in / sign up and 05 Settings: Account; back-end modules `identity` and `email` |
| Design problem | "Four ways to become authenticated: GitHub, Google, email with password, and an existing session. Each exchange differs, all of them must end in one session, and a fifth must not change the session code." (proposal §6.4) |
| Chosen pattern | Strategy with Template Method |
| Requirements sources | `SWE5006 Project Proposal - Lumen.docx`; `docs/ARCHITECTURE.md`, `docs/SLICES.md`, `docs/ONBOARDING.md`, `docs/CODING_STANDARDS.md`, `CODEOWNERS` |
| Notation references | `03a AM.pdf` — «boundary» / «control» / «entity», attributes on entity classes only; `04a DM.pdf` — transition strategies and design activities |
| Deliverables | Seven figures and thirteen tables across four sections: the overview, the module design, the API, and the effort estimate |
| Submitted by | LU YI |
| Date | 8 September 2026 |

### Declaration on the Use of AI Tools

I used Claude (Opus) to render the UML figures from the model I specified and to draft the accompanying explanatory text. I reviewed and revised the models against the SWE5006 Project Proposal and the repository's own documents (`ARCHITECTURE.md`, `SLICES.md`, `ONBOARDING.md`, `CODING_STANDARDS.md`, `CODEOWNERS`), and I am responsible for the content and quality of the submitted work.

### Contents


| Section | What it holds |
|---|---|
| §1 Overvie | What this document delivers, the overall module design of UC1 (Figure 1), and every open item |

| Section | What it holds |
|---|---|
| §2 Module design | Each module taken apart: `identity` in §2.1–§2.5 (Figures 2–7), `email` in §2.6 |
| §3 API | The HTTP interface those modules expose: conventions, four groups of endpoints, and the closed set of failures |
| §4 Effort estimate | The 10 person-days, by proposal task and by sprint |


| Figure | Diagram | Section |
|---|---|---|
| 1 | Module structure of UC1 inside the monolith | §1.2 |
| 2 | Analysis class diagram — UC1 Identity | §2.1 |
| 3 | Analysis sequence diagram — sign in with email and password | §2.1 |
| 4 | Analysis sequence diagram — sign in with an OAuth provider | §2.1 |
| 5 | Design class diagram — Strategy with Template Method | §2.4 |
| 6 | Design sequence diagram — password login | §2.4 |
| 7 | Design sequence diagram — GitHub OAuth login | §2.4 |


Every figure is drawn by `diagrams/gen_diagrams.py` beside this document and exists twice: as the 240 dpi `.png` embedded here, and as an `.svg` that zooms without loss.

# 1. Overview

## 1.1 What this document delivers

This document designs UC1 — sign up, sign in and manage the account. It starts from the overall module structure of the slice (§1.2), takes each module apart in turn (§2), states the HTTP interface those modules expose (§3), and ends with the effort estimate (§4).

Two things it deliberately does not do. It does not design the `email` module's internals, which have no owner decided yet (§2.6). And it produces no code: this round is documents only, and `apps/server/src/modules/identity/` still holds only its README and an empty `index.ts`.

## 1.2 Overall module design

UC1 is one vertical slice holding two modules. Both live inside `apps/server/src/modules/identity/`, in one process against one Postgres database, calling each other in-process rather than over a network.

**Figure 1 — Module structure of UC1 inside the monolith**
Lumen — one process, one Postgres database, modules calling each other in-process.

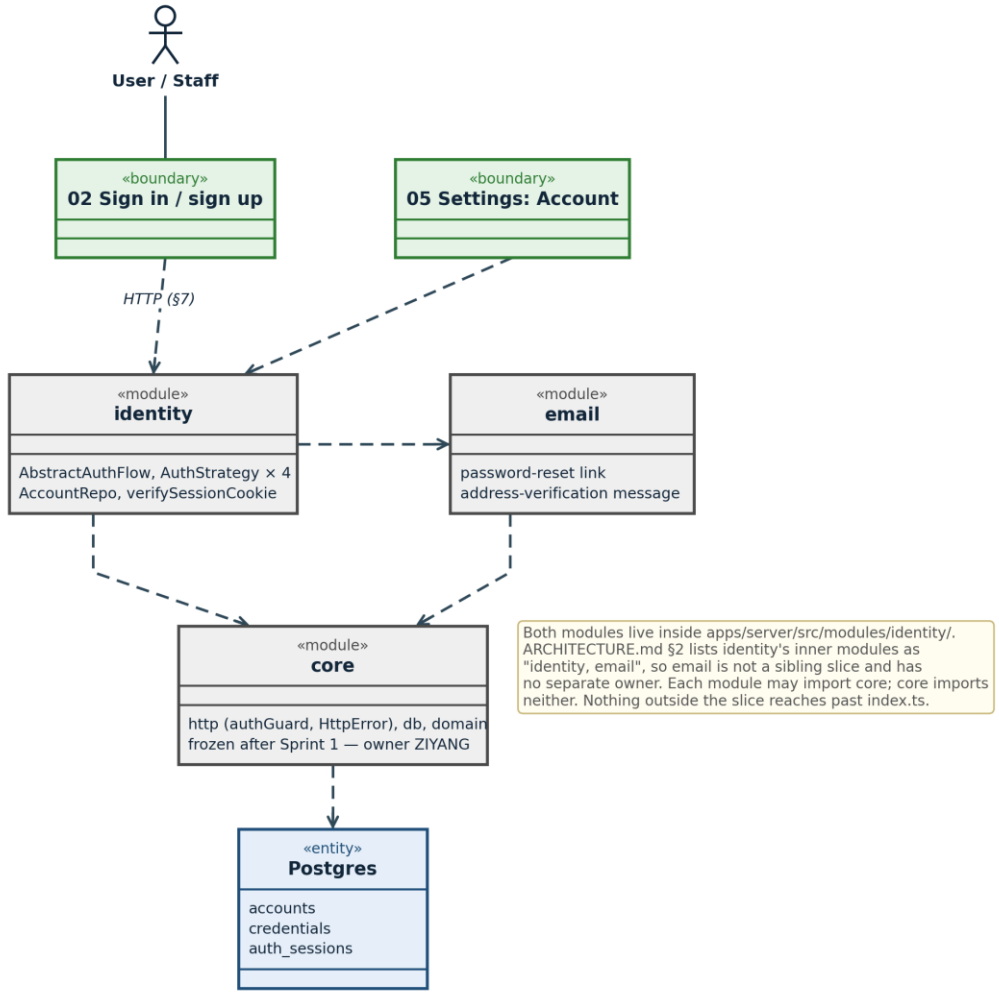



*Figure 1 — Module structure of UC1 inside the monolith*

| Module | What it owns | May import | Taken apart in |
|---|---|---|---|
| `identity` | Authentication by all four routes (password, existing session, GitHub, Google), account records, and login sessions. | `core` only | §2.1–§2.5 |

| Module | What it owns | May import | Taken apart in |
|---|---|---|---|
| | Exposes `verifySessionCookie` so that the auth guard in `core` can resolve a request to a `Principal` without importing this slice. | | |
| `email` | Outbound mail for the identity flows: the password-reset link and the address-verification message. Called in-process by `identity`; it publishes no HTTP endpoints of its own and owns no tables in this round. | `core` only; called by `identity` | §2.6 |

*Table 1 — the two modules of UC1 and what each owns.*

Three things the figure and the table fix:

- **`email` is not a sibling slice.** `ARCHITECTURE.md` §2's module table lists identity's inner modules as "identity, email", so both live under the one slice directory. There is no `modules/email` directory and no second owner.
- **The boundary rule holds for both.** Each module may import `core`; `core` imports neither; nothing outside the slice reaches past `identity/index.ts` (`CODING_STANDARDS.md`). The third transition strategy in §2.2 is what makes real session verification reach `core` without breaking that rule.
- **Only `identity` has a design problem.** Its four authentication routes are what the pattern in §2.3 exists to solve.

Exactly two things cross the slice boundary outwards: the HTTP surface in §3, and the single exported function `verifySessionCookie` described in §2.5. Nothing else.

## 1.3 Sources, assumptions and open items

Two documents govern this slice, and they do not fully agree.

- **The official Project Proposal** (`SWE5006 Project Proposal - Lumen.docx`) — binding for grading. It fixes UC1's scope, its back-end modules, the effort (10 person-days per member, 50 in total), and the required deliverable shape.
- **The repository's own documents** (`ARCHITECTURE.md`, `ONBOARDING.md`, `SLICES.md`, `CODEOWNERS`), written by ZIYANG when scaffolding — what the team works against day to day.

Everything this design decided for itself, and every question it cannot answer alone, is collected below rather than left as remarks scattered through the later sections. The first four are decisions; the last six need someone else.

**Decisions taken here**

- **`auth_sessions`, not `sessions`.** `core/db/schema.sql` already has a `sessions` table meaning an agent session (UC3, tied to a `project_id`). The 2026-09-07 contract rewrite renamed that table, but the concept collision stands, so this slice's login sessions get a distinctly named table (§2.5) and the analysis model names the class `LoginSession` (§2.1).
- **One failure code for two failures.** `invalid_credentials` covers both "no such account" and "wrong password" deliberately; distinguishing them would let an unauthenticated caller enumerate registered addresses. The same reasoning makes `POST /api/auth/password/forgot` answer `204` unconditionally.
- **Staff routing is a client concern.** There is no separate admin login. Every `AuthStrategy` and `AbstractAuthFlow` stays role-agnostic; the client reads `Principal.role` and redirects (§2.1).
- **Other sessions survive a password change.** `POST /api/account/password` revokes nothing else. Session revocation is a security feature UC1 does not claim, and inventing it here would put an unreviewed promise in the contract.

**Open — needs someone else**

- **The two source documents disagree on the `email` module.** The proposal's use-case table (§6.1) lists UC1's back-end modules as "identity, email", while the repository scaffolds only `modules/identity`. Settled since, by `ARCHITECTURE.md` §2, which lists identity's *inner* modules as "identity, email" — so `email` lives inside the identity folder with no separate owner, as §1.2 records. What remains open is that nobody has scaffolded it, so the password-reset endpoints of §3.5 are designed but not yet implementable.
- **The two source documents disagree on the sprint calendar.** The proposal's §8.1 runs Sprint 1 from 1–14 September and Sprint 4 to 3 November; `ONBOARDING.md` compresses the same four sprints into 9-2→9-12, 9-15→9-26, 9-29→10-10, 10-13→10-24, and adds a 10-26 defence and an 11-10 final report that the proposal does not mention. §4.2 uses `ONBOARDING.md`'s dates because that is the calendar the team is actually operating on, but ZIYANG needs to pick one.
- **The error shape does not match what `core` is building.** The accepted core implementation — branch `core/http-fastify`, PR #5, not yet merged into `main` — defines a single `HttpError` class with four factories (`badRequest()`, `unauthorized()`, `forbidden()`, `notFound()`, for 400/401/403/404) and serialises every failure as `{ error: string }`, where `error` carries the human-readable message. There is no machine-readable code field, and no `409` or `500` factory. Either §3.1 folds onto the single field core already has, or `core/http/httpError.ts` — frozen core — takes a reviewed edit. Undecided; it needs ZIYANG.

- **`core` must accept the session-verifier port.** The third transition strategy in §2.2 and its consequence in §2.5 need a small change to `core/http/authGuard.ts`, which `CODEOWNERS` assigns to ZIYANG and which is frozen after Sprint 1. Nothing in this slice can proceed to real authentication until that change is reviewed.
- **Settings → Account has an ownership gap.** `SLICES.md` lists "05 Settings → Account" as part of UC1, but `CODEOWNERS` gives the whole `apps/web/src/pages/settings/` directory — currently one `Settings.tsx` — to HE RONG for billing. Proposed fix, for the team to confirm: split `Settings.tsx` into an owner-neutral shell plus per-tab components (`AccountTab.tsx` to identity, `WalletTab.tsx` and `KeysTab.tsx` to billing) and update `CODEOWNERS` to match.
- **Two failure codes are proposed, not agreed.** Every other code in Table 11 comes from a flow described in the proposal or the repository documents. `invalid_body` and `oauth_not_configured` were added so that the Failures column is complete, and they need whoever owns the shared error contract to accept them.

# 2. Module Design

§1.2 gave the two modules and how they call each other. This section takes each one apart. `identity` carries all of the behaviour, so it gets five subsections — its analysis model, the transition strategies that turn that model into a design, the design problem it owns and the pattern chosen for it, the resulting design model, and where all of it lands in the repository (§2.1–§2.5). `email` has one job and no internal variation, so it needs no pattern and gets one (§2.6).

## 2.1 The `identity` module — analysis model

The module in conceptual terms, before any TypeScript exists: boundary, control and entity only, with no pattern names — those belong to §2.3 and §2.4.

**Figure 2 — Analysis class diagram: UC1 Identity**

Lumen — UC1 Identity, analysis model. Stereotypes «boundary» / «control» / «entity» per 03a AM.pdf; attributes are shown for «entity» classes only.

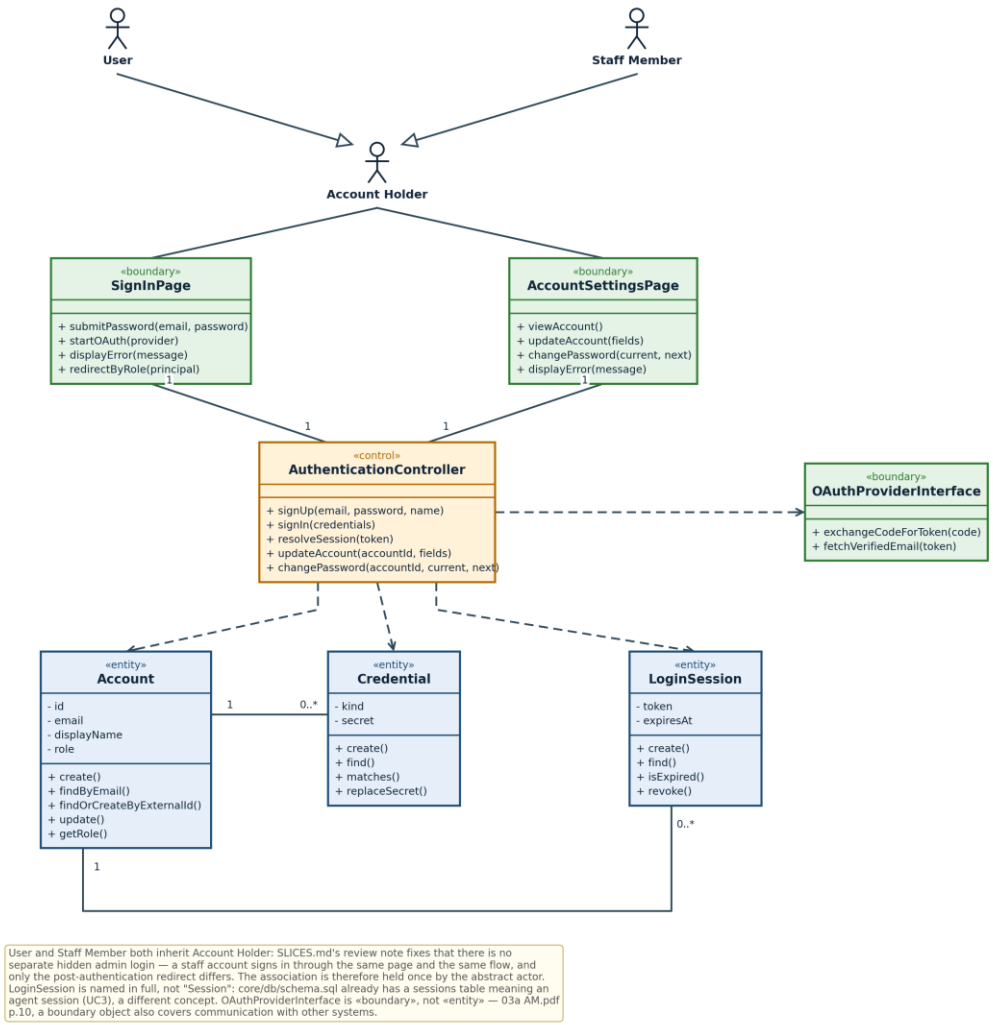



*Figure 2 — Analysis class diagram: UC1 Identity*

| Class | Stereotype | Responsibility | Why it is here |
|---|---|---|---|
| `SignInPage` | «boundary» | Takes an email and password, or starts an OAuth exchange; redirects by role once a principal comes back | Page 02 in `SLICES.md` |
| `AccountSettingsPage` | «boundary» | Views and updates the account, and changes the password | Page 05, Account tab |

| Class | Stereotype | Responsibility | Why it is here |
|---|---|---|---|
| `AuthenticationController` | «control» | The one control class for UC1: sign up, sign in, resolve an existing session, update the account | One control class per use case, `03a AM.pdf` p.21 |
| `OAuthProviderInterface` | «boundary» | Exchanges an authorisation code for a token, then for a verified email address | An external system is a boundary object, not an entity — `03a AM.pdf` p.10 |
| `Account` | «entity» | The person: identity, display name and role | `accounts` already exists in `core/db/schema.sql` |
| `Credential` | «entity» | One way one account can prove itself — a password hash, or a link to an external provider account | An account may hold several; adding a fifth kind must not disturb the others |
| `LoginSession` | «entity» | A live login, its opaque token and its expiry | See the naming decision in §1.3 |

*Table 2 — analysis classes. Attributes are shown for «entity» classes only, per `03a AM.pdf`; operations are derived from the messages each class receives in Figures 3 and 4.*

Two modelling decisions worth defending, both visible in Figure 2:

- **`User` and `Staff Member` both inherit the abstract actor `Account Holder`.** `SLICES.md`'s review note is explicit that there is no separate hidden admin login: a staff account signs in through the same page and the same flow, and only the post-authentication redirect differs. Holding the association once, on the abstract actor, is what says that in the model.
- **`LoginSession` is named in full, not `Session`.** `core/db/schema.sql` already has a `sessions` table, and it means an *agent* session (UC3, tied to a `project_id`) — a different concept from a login session. §1.3 records the collision; §2.5 names the table `auth_sessions` to avoid it.

## Sign in with email and password

**Figure 3 — Analysis sequence diagram: sign in with email and password**
Lumen — UC1 Identity, analysis model. Stereotypes «boundary» / «control» / «entity» per 03a AM.pdf;
attributes are shown for «entity» classes only.
One code for both failure branches: the response must not distinguish "no such account" from "wrong password".

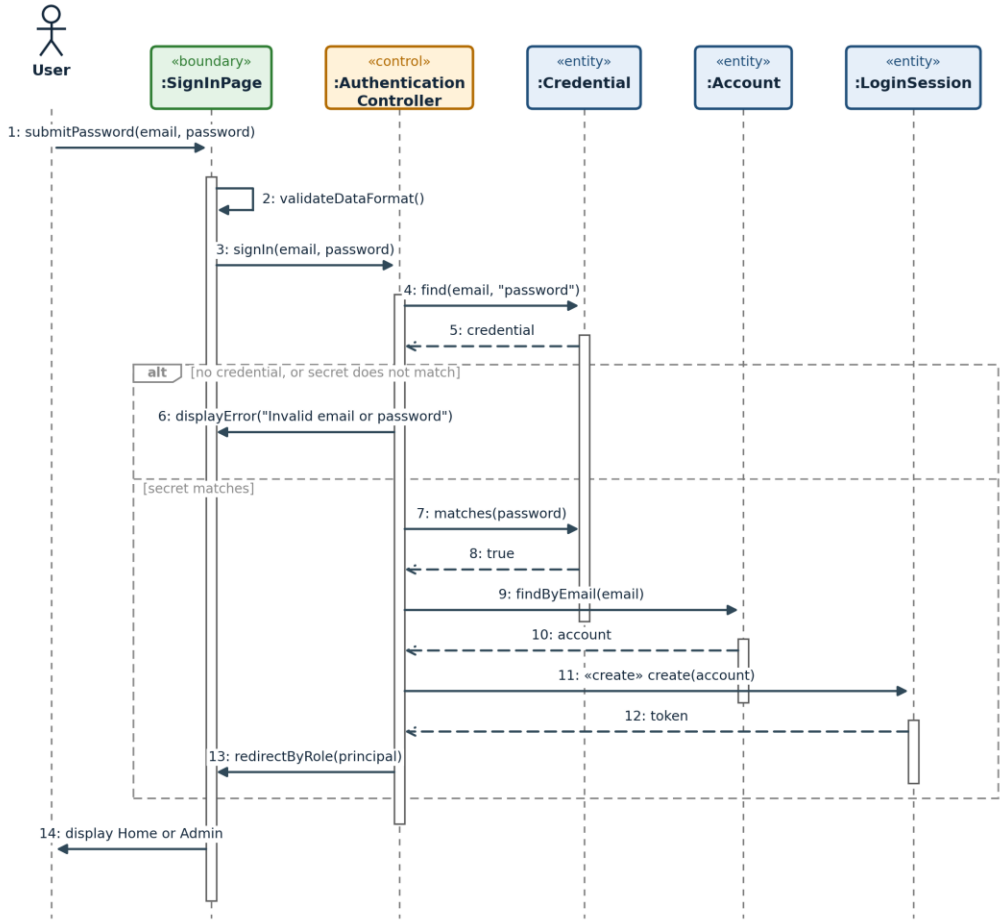



*Figure 3 — Analysis sequence diagram: sign in with email and password*

Both failure branches — no such credential, and a secret that does not match — collapse into one message. The response must not let a caller distinguish “no such account” from “wrong password”; §3.6 carries the same rule into the HTTP contract as a single `invalid_credentials` code.

## Sign in with an OAuth provider

**Figure 4 — Analysis sequence diagram: sign in with an OAuth provider**
Lumen — UC1 Identity, analysis model. Stereotypes «boundary» / «control» / «entity» per 03a AM.pdf; attributes are shown for «entity» classes only.
Messages 11 onwards are identical to Figure 3 from message 11: the two flows differ only in how the email is verified, and converge on one LoginSession. That convergence is the design problem of §4.

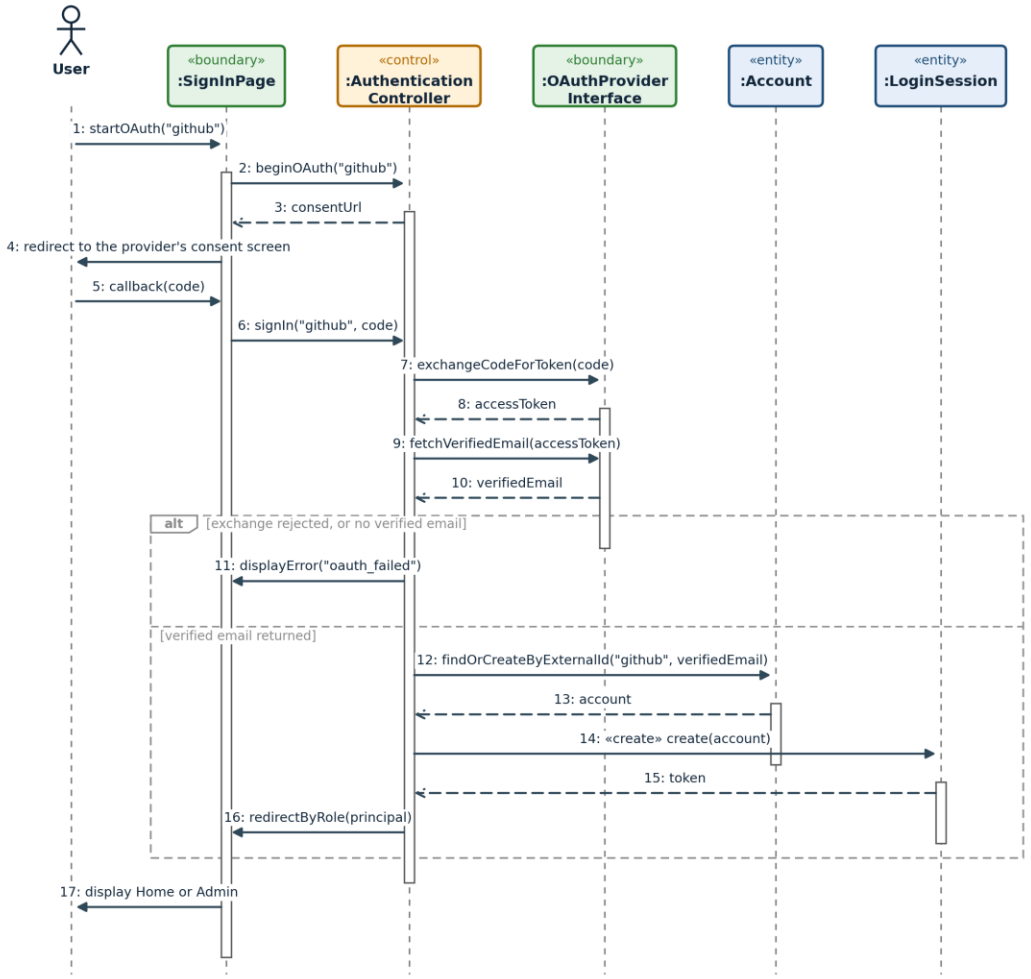



*Figure 4 — Analysis sequence diagram: sign in with an OAuth provider*

Figure 4 is the alternative flow of the same use case, and comparing it with Figure 3 states the design problem in one line: **the two flows differ only in how the email address is verified, and from `LoginSession.create` onward they are identical.** GitHub and Google differ from each other only in which endpoints are called. That convergence — four different beginnings, one shared ending — is what §2.3 has to solve.

## 2.2 The `identity` module — transition strategies

`04a DM.pdf` p.36 defines a transition strategy as a decision about one design concern, usually driven by a non-functional requirement, that constrains how a use

case moves from analysis to design; a collection of them is what makes the resulting design uniform. p.37 fixes the shape each one is written in, and the three below follow it. They come before §2.4 because the design model is their consequence.

### Programming language and standard library

| | |
|---|---|
| **Design concern** | Which language, libraries and conventions the analysis objects become |
| **Non-functional requirement** | Maintainability — "the domain layer has no dependency on the agent library, the web framework or the database driver"; Security — password secrets must not be comparable in variable time |
| **Impacted analysis objects** | `Credential`, and the `matches()` operation on it |
| **Impacted use cases** | UC1, password sign-in and password change |
| **Impacted static structure** | One entity operation, `Credential.matches()` |
| **Impacted dynamic interactions** | Message 7 of Figure 3 |
| **Resulting design objects** | `PasswordStrategy`, holding a private `scryptCompare()` |
| **Resulting static structure** | Strict TypeScript; operations carry parameter and return types, and every asynchronous operation returns `Promise<T>`. Hashing uses Node's built-in `crypto.scrypt` — no new dependency enters the slice. Email lookups are normalised before comparison |
| **Resulting dynamic interactions** | Messages 3–8 of Figure 6 |
| **Justification** | `crypto.scrypt` ships with the runtime the project already requires (Node 22), so the "no dependency on a third-party library" clause is satisfied by not adding one. `crypto.timingSafeEqual` under `scryptCompare()` keeps the comparison constant-time, so a failed sign-in leaks nothing about how nearly it succeeded |

### Persistence

| | |
|---|---|
| **Design concern** | How analysis entities become rows, and which class is allowed to know that |
| **Non-functional requirement** | Maintainability — the domain layer has no |

| | |
|---|---|
| | dependency on the database driver |
| **Impacted analysis objects** | `Account`, `Credential`, `LoginSession` |
| **Impacted use cases** | UC1 in full |
| **Impacted static structure** | All three entity classes and their `create()` / `find()` operations |
| **Impacted dynamic interactions** | Every message in Figures 3 and 4 that reaches an entity |
| **Resulting design objects** | One repository class, `AccountRepo`, plus the tables `credentials` and `auth_sessions` (§2.5) |
| **Resulting static structure** | The entity operations of the analysis model collapse into `AccountRepo`, which is **the only class in the slice that issues SQL**. The strategies and the flow depend on it and never on the driver. `accounts` already exists in `core/db/schema.sql`; this slice adds the other two |
| **Resulting dynamic interactions** | In Figures 6 and 7 every database access is a message to `:AccountRepo`, which then talks to `:Postgres`. No other lifeline touches the database |
| **Justification** | One repository is what makes the “no dependency on the database driver” clause checkable by reading imports, and it is what lets `PasswordStrategy` be unit-tested against a stub repository rather than a live Postgres. The token is stored as `token_hash`, never in clear, so a database dump does not yield usable session cookies |

## Layering and the slice boundary

| | |
|---|---|
| **Design concern** | How the shared HTTP layer authenticates a request without importing this slice |
| **Non-functional requirement** | Maintainability and Extensibility — `CODING_STANDARDS.md`: `core` may never import a slice; Security — every request must pass real verification once it exists |
| **Impacted analysis objects** | `AuthenticationController.resolveSession()`, `LoginSession` |
| **Impacted use cases** | UC1, and every other use case, because every request passes the guard |
| **Impacted static structure** | `core/http/authGuard.ts`, which today trusts a stub header through `principalFromHeader` |

| | |
|---|---|
| **Impacted dynamic interactions** | The resolve-session flow, and the first step of every other request in the system |
| **Resulting design objects** | `SessionStrategy` and the function `verifySessionCookie` it backs |
| **Resulting static structure** | `authGuard.ts` gains a module-level port, `registerSessionVerifier(verify)`, and calls the registered verifier instead of the stub. `identity/index.ts` exports `verifySessionCookie`. `apps/server/src/main.ts` — the composition root, which already imports every slice — adds one line at bootstrap. Neither `core` nor `identity` imports the other. The three lines this costs are in §2.5 |
| **Resulting dynamic interactions** | Unchanged for callers: the guard still returns a `Principal` or `null`. Only the implementation behind it moves |
| **Justification** | Inverting the dependency is the only way to satisfy both rules at once, and it is what `ONBOARDING.md` describes when it says 「陆一的真登录上线之前，大家先用桩账号开发；他一上线，全员切过去」 — *before LU YI's real login ships everyone develops against stub accounts, and the moment it ships everyone switches over*. A port makes that switch one line in the composition root instead of an edit to every caller. It does need a small reviewed change to `core`, which is frozen after Sprint 1 — see §1.3 |

## 2.3 The `identity` module — design problem and pattern

The design problem this slice owns, quoted from the Project Proposal §6.4:

> Four ways to become authenticated: GitHub, Google, email with password, and an existing session. Each exchange differs, all of them must end in one session, and a fifth must not change the session code.

The problem statement contains its own solution shape. It names one thing that varies — the exchange — and two that must not: the session that every route ends in, and the code that issues it. A pattern that fixes the algorithm once and lets only the varying step be substituted is exactly what is being asked for.

**Chosen: Strategy with Template Method.**

- `AbstractAuthFlow.authenticate()` is the **template method**. It fixes the three-step algorithm exactly once — find the strategy whose `kind` matches, call `verify()`, call `onAuthenticated()` — and no subclass overrides it.
- `AuthStrategy` is the **strategy interface**. Each concrete strategy supplies only `verify()`, the one step that actually differs between the four routes. `PasswordStrategy` compares a scrypt hash; `SessionStrategy` looks up a token; `GithubStrategy` and `GoogleStrategy` exchange an authorisation code for a verified email.

Three consequences, each answering a clause of the problem statement:

| The problem says | The pattern answers |
|---|---|
| "each exchange differs" | Each exchange is one `verify()` in one class, and nothing else in the module needs to know which one ran |
| "all of them must end in one session" | `onAuthenticated()` runs from a single place inside the template method, for every route. It is structurally impossible for a route to skip it |
| "a fifth must not change the session code" | A fifth way in is one new class realising `AuthStrategy`, registered in `strategies()`. `authenticate()` and `onAuthenticated()` are untouched |

*Table 3 — how the chosen pattern answers the design problem.*

**Considered and rejected.** The proposal's §6.4 names candidate patterns as a hypothesis, not a conclusion, so the three nearest alternatives are compared here rather than assumed away.

- **Strategy alone, without the template method.** Each of the four strategies implements the whole flow. The varying step is isolated, but nothing then guarantees that "all of them must end in one session": every strategy has to remember to call `onAuthenticated()`, and one that forgets is a login route issuing no session. The fifth way in has to remember it too. The second clause of the problem statement asks for exactly that guarantee, so Strategy on its own does not answer it.
- **A factory.** It produces the right handler for a `kind`, but it answers only *which one*, never *what stays fixed once one is chosen*. The lookup in `strategies()` already answers *which one*, so a factory above it adds no constraint that the problem statement asks for.
- **Chain of responsibility.** Four handlers try in turn until one claims the request. It would run, but the semantics are wrong: `kind` is stated by the caller, so nothing needs probing. It also makes "wrong password" hard to

separate from “no such kind”, which contradicts §3.6, where `invalid_credentials` and `unsupported_kind` are deliberately distinct codes.

A fourth consequence is not in the problem statement but matters for the Definition of Done: each `verify()` is a unit that can be tested on its own against a stub `AccountRepo`, rather than through the whole login path.

## 2.4 The `identity` module — design model

**Figure 5 — Design class diagram: Strategy with Template Method**
Lumen — UC1 Identity, design model. Design problem: four ways to become authenticated must end in one session, and a fifth must not change the session code. Pattern: Strategy with Template Method.

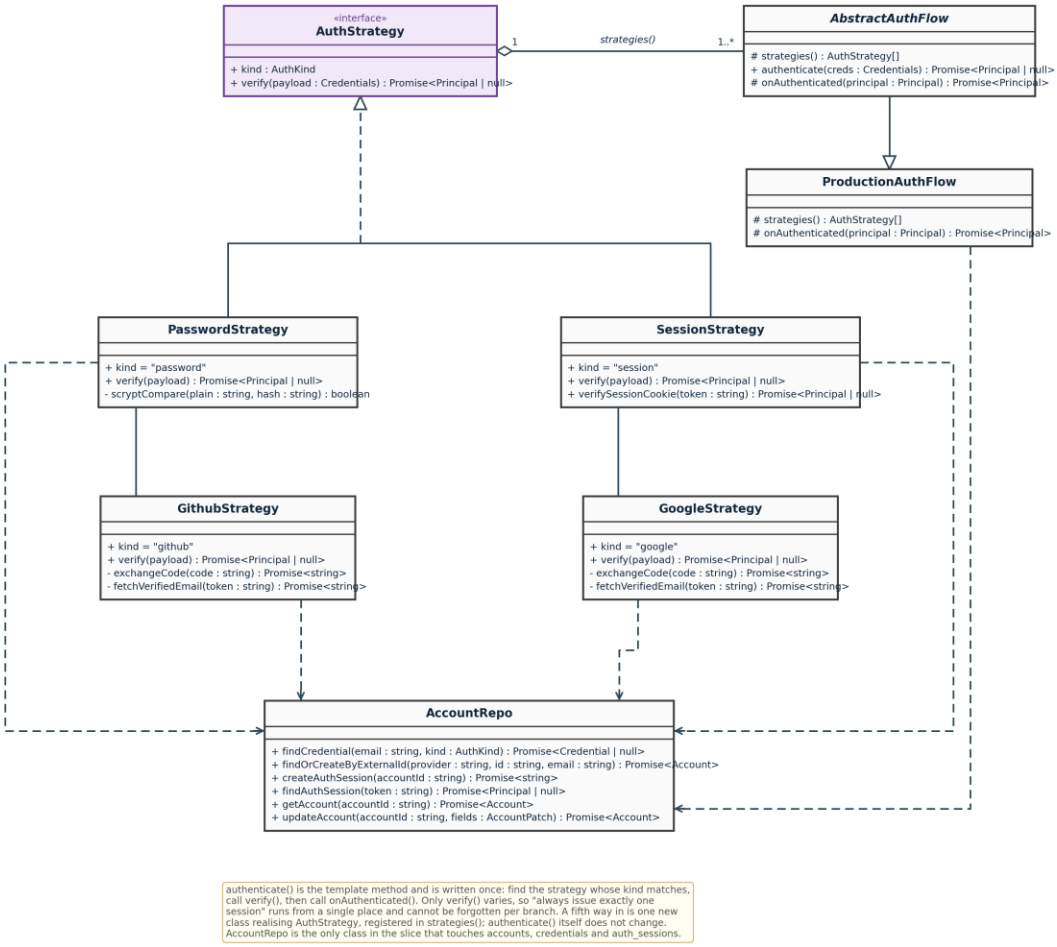



*Figure 5 — Design class diagram: Strategy with Template Method*

Read together with §2.2, the table below is the audit trail from Figure 2 to Figure 5: every analysis class is accounted for, and no design class appears without a reason for existing.

| Analysis (Figure 2) | Design (Figure 5) | What moved it |
|---|---|---|

| Analysis (Figure 2) | Design (Figure 5) | What moved it |
|---|---|---|
| `AuthenticationController` | `AbstractAuthFlow` + `ProductionAuthFlow` | The pattern, §2.3 — the control class splits into a fixed algorithm and the concrete flow that supplies the strategy list |
| `Credential.matches()` | `PasswordStrategy.verify()` with a private `scryptCompare()` | Programming language, §2.2 |
| `OAuthProviderInterface` | `GithubStrategy` and `GoogleStrategy`, each with `exchangeCode()` and `fetchVerifiedEmail()` | Programming language, §2.2; and the pattern, §2.3 |
| `LoginSession` | `AccountRepo.createAuthSession()` / `findAuthSession()` over the `auth_sessions` table | Persistence, §2.2 |
| `Account`, `Credential` | `AccountRepo`, the only class issuing SQL | Persistence, §2.2 |
| `AuthenticationController.resolveSession()` | `SessionStrategy`, reused as `verifySessionCookie` behind the guard's port | Layering, §2.2 |

*Table 4 — the analysis model mapped onto the design model.*

## Password login

**Figure 6 — Design sequence diagram: password login**

Lumen — UC1 Identity, design model. Design problem: four ways to become authenticated must end in one session, and a fifth must not change the session code. Pattern: Strategy with Template Method.

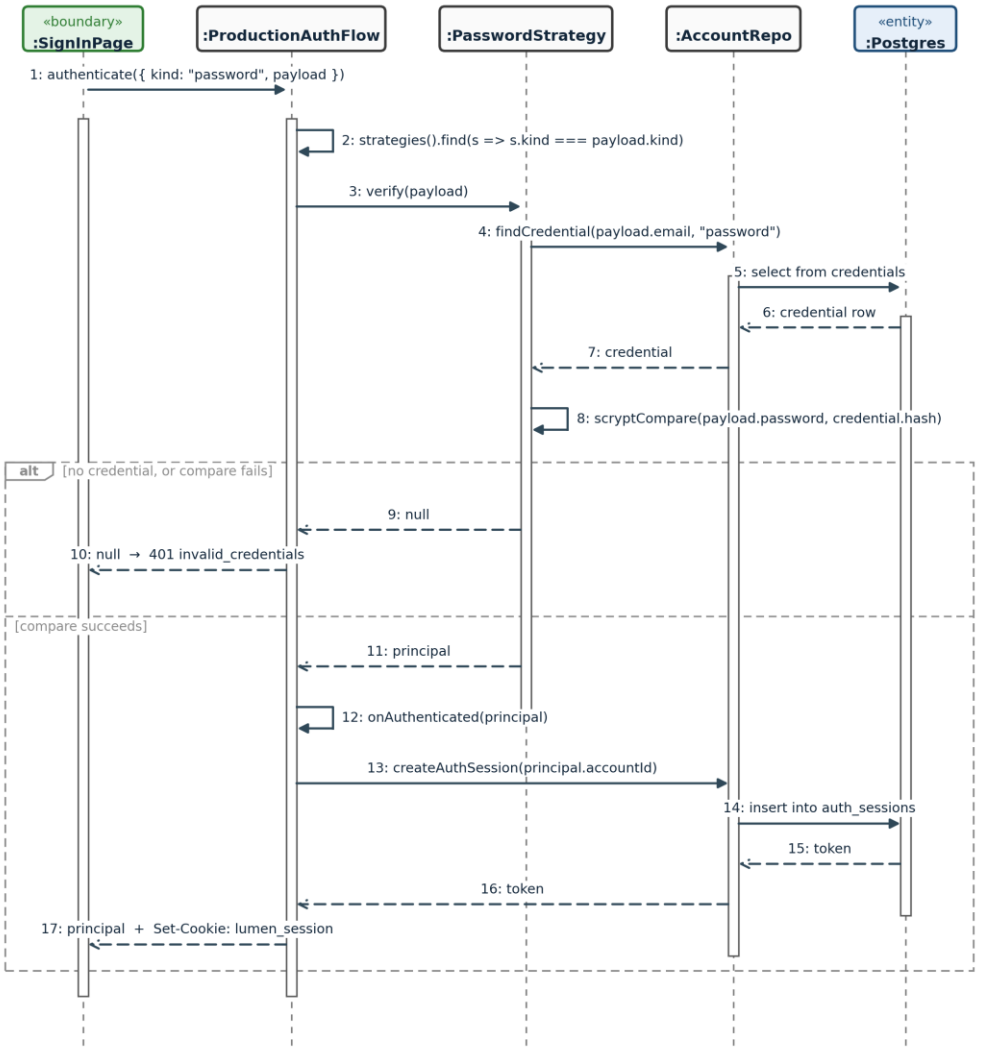



*Figure 6 — Design sequence diagram: password login*

## GitHub OAuth login

**Figure 7 — Design sequence diagram: GitHub OAuth login**
Lumen — UC1 Identity, design model. Design problem: four ways to become authenticated must end in one session, and a fifth must not change the session code. Pattern: Strategy with Template Method. Messages 12 to 18 are byte for byte the messages 11 to 17 of Figure 6. Only verify() differs, and the shared ending runs from onAuthenticated() in the base class — which is what the pattern was chosen for.

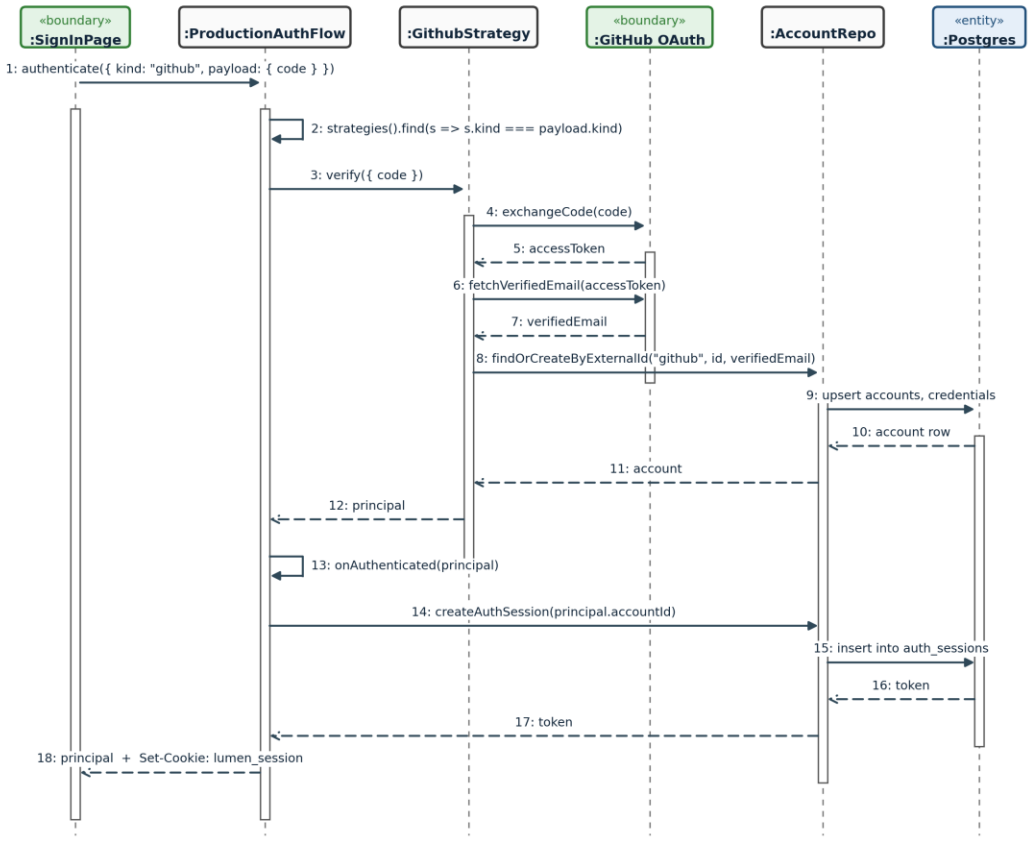



*Figure 7 — Design sequence diagram: GitHub OAuth login*

Messages 12 to 18 of Figure 7 are, message for message, messages 11 to 17 of Figure 6. Only `verify()` differs. The shared ending runs from `onAuthenticated()` in the base class, which is the whole point of the pattern and the reason Figure 7 is worth printing next to Figure 6 rather than described in a sentence.

Google login and the session check are the same two diagrams again with `GoogleStrategy` or `SessionStrategy` substituted, so they are not repeated.

## 2.5 The `identity` module — files, storage and wiring

Where the design model of §2.4 lands in the repository. Everything is inside `apps/server/src/modules/identity/` except the schema migration and the one line in the composition root, both of which touch `core`-owned files and therefore need ZIYANG's review.

## Files

The folder currently holds a README and an empty `index.ts` — the 2026-09-07 contract rewrite removed the earlier scaffolding — so every file below is new.

| File | Responsibility | Sprint |
|---|---|---|
| `authStrategy.ts` | The `AuthStrategy` interface, the `Credentials` type and `AbstractAuthFlow` | 1 |
| `passwordStrategy.ts` | `PasswordStrategy`. Hashing via Node's built-in `crypto.scrypt` (§2.2) — normalised email lookup, timing-safe compare | 1 |
| `sessionStrategy.ts` | `SessionStrategy`. Looks up `auth_sessions` by opaque token and rejects it if expired. Used both as an `AuthStrategy` (`kind: 'session'`) and directly by `verifySessionCookie` | 1 |
| `accountRepo.ts` | The only file in the slice touching `accounts`, `credentials` and `auth_sessions` (§2.2) | 1 |
| `productionAuthFlow.ts` | The concrete `AbstractAuthFlow`. `strategies()` returns password and session from Sprint 1, plus github and google once Sprint 3 lands — the template method does not change either way. `onAuthenticated` persists an `auth_sessions` row | 1 |
| `githubStrategy.ts` | `GithubStrategy`. Exchanges an OAuth code for an access token, fetches the verified primary email, calls `accountRepo.findOrCreateByExternalId` | 3 |
| `googleStrategy.ts` | The same shape against Google's OAuth/OpenID endpoints | 3 |
| `identity.test.ts` | Unit tests written against Figure 5: each `verify()` alone, `AccountRepo` against a test database, and `authenticate()` against stub strategies | 1 and 3 |
| `index.ts` | The public index. Adds `verifySessionCookie`, `ProductionAuthFlow`, and whatever the sign-in page needs at the HTTP layer | 1 |

*Table 5 — file-by-file plan. Nothing outside `modules/identity` imports any of these except through `index.ts` (`CODING_STANDARDS.md`).*

## Storage

A change to the core-owned `apps/server/src/core/db/schema.sql`, so per `CODEOWNERS` it goes through ZIYANG's review even though the tables hold this slice's data.

```
-- identity slice — LU YI. Adds credential storage and login sessions.
-- Named auth_sessions, not sessions: the existing `sessions` table is
an
-- agent/project session (UC3), a different concept from a login sessio
n.

create table if not exists credentials (
  id                uuid primary key default gen_random_uuid(),
  account_id        uuid not null references accounts(id),
  kind              text not null check (kind in ('password', 'github',
 'google')),
  password_hash     text,
  external_provider text,
  external_id       text,
  created_at        timestamptz not null default now(),
  unique (kind, external_provider, external_id)
);

create table if not exists auth_sessions (
  id         uuid primary key default gen_random_uuid(),
  account_id uuid not null references accounts(id),
  token_hash text not null unique,
  created_at timestamptz not null default now(),
  expires_at timestamptz not null
);
```

## Wiring real session verification into `core`

The third transition strategy of §2.2 costs exactly three lines.

1. `core/http/authGuard.ts` gains `registerSessionVerifier(verify: (token: string) => Promise<Principal | null>)` and calls the registered verifier in front of the Sprint 1 stub `principalFromHeader`.
2. `identity/index.ts` exports `verifySessionCookie`, a thin wrapper around `SessionStrategy.verify`.
3. `apps/server/src/main.ts` adds `registerSessionVerifier(verifySessionCookie)` at bootstrap.

`main.ts` is the composition root and already imports every slice directly, so it is allowed to import both sides; it is not itself `core` and not a slice.

## 2.6 The `email` module

| | |
|---|---|
| **Responsibility** | Two outbound messages, both triggered by `identity` and by nothing else: the password-reset link, and the address-verification message sent after signup |
| **Callers** | `identity` only, in-process. `POST /api/auth/password/forgot` (§3.5) hands it an address; signup hands it a new account's address |
| **Interface** | Called through the slice's own public index, not over HTTP. It publishes no endpoints |
| **Data** | None. It owns no tables this round; the reset token lives in `identity`'s own storage, and the module is handed the address and the token to send |
| **Design problem** | None. There is one way to send a message and no variation to absorb, so no pattern applies |
| **Status** | **Not designed this round.** Nobody has scaffolded the module and no owner has been assigned — see §1.3. The two endpoints that call it are designed in §3.5 and become implementable the moment it exists |

*Table 6 — the `email` module.*

Designing its internals now would mean guessing at a mail transport, a template mechanism and a retry policy that nobody has agreed, and then handing the eventual owner a design they did not write. What this document does commit to is the boundary: `email` is called in-process by `identity`, imports only `core`, and publishes nothing of its own.

# 3. API

The HTTP interface the two modules of §1.2 expose to the rest of the system. `identity` publishes all of it; `email` publishes none, and appears here only as what stands behind §3.5.

JSON over HTTP. The server framework is not chosen on `main` yet, and the other four slices have not published their own contracts, so this section is written as this slice's own commitment: the endpoints `identity` will expose, complete enough for the sign-in page and the Settings → Account tab to be built against them, and for another owner to read without asking. Wiring waits on whatever `core` exposes (§1.3), but the contract does not.

## 3.1 Conventions

**Session cookie.** `lumen_session` — `httpOnly`, `Secure`, `SameSite=Lax`, an opaque token that maps to `auth_sessions.token_hash`. Set on a successful login or signup, refreshed on a successful session check, cleared on logout.

**Error shape.** The shape proposed for `packages/contracts`, which is frozen and so needs the PM's sign-off:

```
interface ApiError {
  readonly error: string;   // machine-readable, e.g. "invalid_credentials"
  readonly message: string; // human-readable
}
```

**This does not match what core will actually provide**, and the mismatch is unresolved — see §1.3. The status codes below are unaffected either way; only the body shape is in question.

**Reading the endpoint tables.** §3.2 to §3.5 take one group of endpoints each and give, per endpoint, how it is called, what a successful call returns, and every failure it can produce. The failure codes are defined once in §3.6; `Principal` is §3.7.

## 3.2 Authentication endpoints

| Method and path | How it is called | Success | Failures |
|---|---|---|---|
| `POST /api/auth/signup` | Body `{ email, password, displayName }`. No cookie required. | `201` `{ principal: Principal }` + `Set-Cookie: lumen_session` | `409 email_taken` `400 invalid_body` |
| `POST /api/auth/login` (password) | Body `{ kind: 'password', email, password }`. Calls `ProductionAuthFlow.authenticate({ kind, payload })`. | `200` `{ principal: Principal }` + `Set-Cookie: lumen_session` (refreshed) | `401 invalid_credentials` `400 invalid_body` `400 unsupported_kind` (for `kind: 'github'` / `'google'`) |
| `POST /api/auth/login` (session) | Body `{ kind: 'session' }` plus the `lumen_session` cookie. Resolved by `SessionStrategy`. | `200` `{ principal: Principal }` + `Set-Cookie: lumen_session` (refreshed) | `401 invalid_credentials` — cookie missing, unknown, or past `expires_at` |
| `POST /api/auth/logout` | Empty body, `lumen_session` cookie. | `204`, no body. Clears the cookie and deletes its `auth_sessions` row. | None. Logging out without a valid session still returns `204` — the call is idempotent. |
| `GET /api/auth/me` | No body, `lumen_session` cookie. What the client calls on page load, and what the role-based | `200` `{ principal: Principal }` | `401 not_authenticated` |

| Method and path | How it is called | Success | Failures |
|---|---|---|---|
| | `/admin` redirect reads. | | |

*Table 7 — authentication endpoints. All five run through `ProductionAuthFlow.authenticate()` (Figure 6).*

## 3.3 OAuth endpoints (Sprint 3)

OAuth needs a browser redirect rather than a JSON round trip, which is why `kind: 'github'` and `kind: 'google'` are rejected at `POST /api/auth/login` and get this start/callback pair instead. Both callbacks answer with a redirect, so their failures arrive as a query parameter on `/signin`, not as a JSON error body.

| Method and path | How it is called | Success | Failures |
|---|---|---|---|
| `GET /api/auth/oauth/github/start` | Browser navigation. No body, no cookie. | `302` to GitHub's consent screen. `GITHUB_OAUTH_CLIENT_ID` from `.env`, scope: verified email only. | `500 oauth_not_configured` — client id or secret missing from `.env` |
| `GET /api/auth/oauth/github/callback?code=…` | GitHub redirects the browser here with `code`. Calls `authenticate({ kind: 'github', payload: { code } })`. | `302` to `/` or `/admin` by role + `Set-Cookie: lumen_session` | `302` to `/signin?error=oauth_failed` — code exchange rejected, or no verified email on the provider account |
| `GET /api/auth/oauth/google/start` | The same shape as the GitHub pair, against Google's OAuth/OpenID endpoints and `GOOGLE_OAUTH_CLIENT_ID` / `GOOGLE_OAUTH_CLIENT_SECRET`. | Same as GitHub. | Same as GitHub. |
| `GET /api/auth/oauth/google/callback?code=…` | | | |

*Table 8 — OAuth endpoints. Both providers run the same `authenticate()` shape with a different `AuthStrategy` (Figure 7).*

## 3.4 Account endpoints

All three require a valid `lumen_session` cookie and all three return `401 not_authenticated` without one, so that row is not repeated per endpoint.

| Method and path | How it is called | Success | Failures |
|---|---|---|---|
| `GET /api/account` | No body, `lumen_session` cookie. | `200 { accountId, email, displayName, role }`, `role` being `'user'` or `'staff'` | `401 not_authenticated` |
| `PATCH` | Body `{ displayName?,` | `200` | `401 not_authenticated` |

| Method and path | How it is called | Success | Failures |
|---|---|---|---|
| `/api/account` | `email? }` — both optional, at least one required. | `{ accountId, email, displayName, role }`, the updated record | `409 email_taken` (the new address is already in use)<br>`400 invalid_body` (empty patch, or malformed address) |
| `POST /api/account/password` | Body `{ currentPassword, newPassword }`, `lumen_session` cookie. | `204`, no body. Other login sessions are deliberately left alive — revoking them is outside UC1's scope. | `401 invalid_credentials` (wrong `currentPassword`)<br>`401 not_authenticated`<br>`400 invalid_body` (new password fails the length rule) |

*Table 9 — account endpoints, behind Settings → Account. The ownership caveat on that page is in §1.3.*

## 3.5 Password-reset endpoints

Both endpoints hand work to the `email` module (§2.6), which is why both are designed here but neither is implementable yet.

| Method and path | How it is called | Success | Failures |
|---|---|---|---|
| `POST /api/auth/password/forgot` | Body `{ email }`. No cookie. Hands the address to the `email` module, which sends the reset link. | `204` **always**, whether or not the address exists — the response must not reveal which addresses are registered. | `400 invalid_body` only (missing or malformed address). A rate limit belongs here but is not designed this round. |
| `POST /api/auth/password/reset` | Body `{ token, newPassword }`. No cookie — the token is the proof. | `204`, no body. | `400 invalid_or_expired_token` (unknown, already used, or past expiry)<br>`400 invalid_body` |

*Table 10 — password-reset endpoints.*

## 3.6 Failure codes

Every failure the endpoints above can return, in one place, so the client pages can be written against a closed set.

| Code | Status | When | Returned by |
|---|---|---|---|
| `invalid_body` | `400` | A required field is missing, or a field fails its format or length rule. Raised before any strategy runs. | signup, both `login` shapes, `PATCH /api/account`, `POST /api/account/password`, both `password/*` |

| Code | Status | When | Returned by endpoints |
|---|---|---|---|
| `invalid_credentials` | `401` | Password sign-in failed, the session cookie is absent, unknown or expired, or `currentPassword` is wrong. **One code covers all of these on purpose** — the response must not distinguish "no such account" from "wrong password" (§2.1) | `POST /api/auth/login`, `POST /api/account/password` |
| `not_authenticated` | `401` | The endpoint needs a signed-in caller and there is no valid session | `GET /api/auth/me`, `GET/PATCH /api/account`, `POST /api/account/password` |
| `unsupported_kind` | `400` | Called with `kind: 'github'` or `'google'`, which go through the OAuth start/callback pair instead | `POST /api/auth/login` |
| `email_taken` | `409` | The address already belongs to another account | `POST /api/auth/signup`, `PATCH /api/account` |
| `invalid_or_expired_token` | `400` | The reset token is unknown, already used, or past its expiry | `POST /api/auth/password/reset` |
| `oauth_failed` | `302` | The provider rejected the code exchange, or the account has no verified email. Not a JSON body — it arrives as `/signin?error=oauth_failed` | both OAuth callbacks |
| `oauth_not_configured` | `500` | The provider's client id or secret is missing from `.env` | both OAuth start endpoints |

*Table 11 — the closed set of identity failures.*

Two absences worth stating. There is **no `403`**: role never gates an identity endpoint, because a staff account signs in through exactly the same flow as anyone else and `/admin` routing is a client-side read of `Principal.role` (§2.1). And there is **no `404`**: unknown paths are `core`'s router concern, not this module's.

## 3.7 `Principal`

The one type every endpoint above returns. Already defined in `core/http/authGuard.ts`, which is `core`-owned:

```
interface Principal {
  readonly accountId: string;
```

```
  readonly role: 'user' | 'staff';
}
```

# 4. Effort Estimate

## 4.1 By proposal task

The proposal fixes this precisely in §8: five members at ten person-days each, fifty in total, split into seven tasks. For LU YI, the 10-day budget lands as:

| Category (proposal task) | Days | This slice's work |
|---|---|---|
| Common: requirements, backlog, sprint plans, shared contract, schema and pipeline (tasks 1–2, split five ways) | 1.6 | The same for every member — no identity-specific content |
| Analysis and design of UC1 (task 3) | 2.0 | Figures 1–7 and the transition strategies in §2.2 |
| Design problem: Strategy with Template Method (task 4) | 1.4 | §2.3 — the problem, the chosen pattern and its justification |
| Build the identity vertical slice (task 5) | 3.4 | The nine files in Table 5, the sign-in page and the Settings → Account tab. **Not started: this round is documents only** |
| Pipeline jobs for identity (task 6) | 0.8 | CI already runs SCA and CodeQL generically; the identity-specific concerns are OAuth secret handling and login-endpoint coverage once DAST has a target (Sprint 2 onward, per `DOD.md`) |
| Integration, hardening, report, presentation (task 7, split five ways) | 0.8 | The same for every member |
| **Total** | **10.0** | |

*Table 12 — effort by proposal task. The 10.0-day category total is the number the proposal grades against.*

## 4.2 By sprint

Mapped onto `ONBOARDING.md`'s calendar — see the date caveat in §1.3.

| Sprint | Dates | Identity work | Days |
|---|---|---|---|
| 1 | 9-2 – 9-12 | Shared-contract share (0.8) + this design document (2.0) + start of build: password and | ~5.0 |

| Sprint | Dates | Identity work | Days |
|---|---|---|---|
| | | session strategies, `AccountRepo`, the migration and the `registerSessionVerifier` wiring (2.2 of the 3.4 build days) | |
| 2 | 9-15 – 9-26 | Rest of build: the sign-in page, the Settings → Account tab (pending the ownership item in §1.3) and the account endpoints (1.2 of the 3.4 build days) | ~1.2 |
| 3 | 9-29 – 10-10 | The design-problem write-up (1.4) and the GitHub / Google OAuth build | ~1.4 |
| 4 | 10-13 – 10-24 | Pipeline jobs (0.8) and the shared integration and report (0.8) | ~1.6 |
| **Total** | | | **~10.0** |

*Table 13 — the weekly shape is illustrative; the fixed number the proposal grades against is the 10.0-day category total in Table 12.*